

Morphic Studio Software Architecture Specification

Document 43 — AI Orchestration Layer & Unified AI Gateway Specification

Version: 1.0

Status: Approved

1. Purpose

This document defines the architecture of the AI Orchestration Layer and the Unified AI Gateway.

Its purpose is to provide a single intelligent coordination layer between Morphe Studio and all AI providers while ensuring consistency, flexibility, observability, and future-proofing.

The rest of the platform interacts with capabilities rather than specific AI models or vendors.

2. AI Philosophy

The AI layer follows these principles:

2.1 Capability-Based Routing

Applications request capabilities, not models.

Examples:

- Generate Story Outline
- Improve Dialogue
- Build Character Profile
- Generate Comic Panel
- Produce Voice
- Analyze Story Consistency
- Create Animation Plan

The Gateway selects the most appropriate provider and model.

2.2 Provider Independence

No application module communicates directly with:

- OpenAI
- Anthropic
- Google
- xAI
- DeepSeek
- Mistral
- Local models

All communication passes through the Unified AI Gateway.

Providers can be added, removed, or replaced without changing application code.

2.3 Context Before Generation

Every AI request should include relevant context assembled from:

- Story Bible
- Creative Memory
- Active project
- Current scene
- Characters
- World information
- User preferences
- Previous AI outputs

The objective is to maximize consistency and reduce hallucinations.

2.4 Human Control

AI recommendations never silently overwrite user-created content.

Creators approve significant changes before they become canonical.

2.5 Explainable Assistance

Where practical, AI responses should include reasoning or references to project context so users understand why recommendations were made.

3. Unified AI Gateway Responsibilities

The Gateway is responsible for:

- Capability routing
 - Provider selection
 - Prompt assembly
 - Context injection
 - Safety checks
 - Response normalization
 - Retry logic
 - Cost tracking
 - Latency monitoring
 - Fallback routing
 - Logging
 - Observability
-

4. AI Capability Catalog

Capabilities are grouped into categories.

Story Intelligence

- Outline generation
- Plot analysis
- Scene planning
- Pacing analysis
- Dialogue refinement

Character Intelligence

- Character creation
- Relationship analysis
- Personality consistency
- Voice refinement
- Character development

World Intelligence

- Lore creation
- Geography
- Governments
- Cultures
- Magic systems

- Technology systems

Visual Intelligence

- Storyboards
- Comic panels
- Character art
- Backgrounds
- Concept art

Audio Intelligence

- Voice synthesis
- Narration
- Sound effects
- Music suggestions

Production Intelligence

- Camera planning
 - Animation planning
 - Scene sequencing
 - Production optimization
-

5. Request Lifecycle

Every AI request follows this sequence:

User Action

↓

Capability Request

↓

Permission Check

↓

Context Assembly

↓

Prompt Construction

↓

Provider Selection

↓

Execution

↓

Response Validation

↓

Normalization

↓

Storage (if applicable)

↓

User Presentation

↓

Feedback Collection

Every stage should be observable and traceable.

6. Provider Routing Strategy

Provider selection considers:

- Capability suitability
- Response quality
- Latency
- Cost
- Availability
- User preferences
- Subscription tier
- Geographic restrictions
- Rate limits

Routing decisions should remain configurable.

7. Prompt Orchestration

Prompt construction combines:

System instructions

+

Capability template

+

Story Bible context

+

Creative Memory

+

Current user request

+

Retrieved assets (if applicable)

+

Output requirements

Prompt templates should be version-controlled and reusable.

8. Safety & Guardrails

Implement safeguards including:

Input validation

Output validation

Prompt injection resistance

Content moderation

Rate limiting

Sensitive data protection

Policy enforcement

Hallucination mitigation where practical

Safety mechanisms should evolve independently of providers.

9. Multi-Model Collaboration

Complex workflows may involve multiple AI models.

Example:

Reasoning model

↓

Story planner

↓

Image generator

↓

Voice synthesizer

↓

Consistency checker

↓

Final assembler

The Gateway coordinates these pipelines transparently.

10. Context Management

Context sources include:

Story Bible

Creative Memory

Character Profiles

World Data

Scene History

Production Timeline

Asset Metadata

User Preferences

Conversation History (where appropriate)

Only relevant context should be supplied to optimize quality and cost.

11. Cost Management

Track:

Token usage

Image generation costs

Audio generation costs

Rendering costs

Provider usage

Project usage

User usage

Support configurable budgets and alerts.

12. Observability

Monitor:

Provider latency

Failure rates

Retry frequency

Fallback usage

Cost trends

Token consumption

Capability popularity

Response quality metrics

Support dashboards, alerts, and analytics.

13. Extensibility

The architecture must support future:

Local models

Enterprise AI deployments

Custom fine-tuned models

Community plugins

User-provided API keys

Private inference servers

Experimental providers

without modifying application code.

14. Acceptance Criteria

The AI orchestration layer is successful if:

- Applications request capabilities instead of providers.
 - AI vendors remain interchangeable.
 - Context is consistently assembled.
 - Costs are measurable.
 - Failures are recoverable.
 - Multi-model workflows are supported.
 - Users retain creative control.
 - AI behavior remains consistent across the platform.
-

Conclusion

The AI Orchestration Layer transforms Morphe Studio from a collection of AI integrations into an intelligent creative operating system. By centralizing capability routing, context assembly, provider abstraction, safety, and observability, the platform gains flexibility, consistency, and resilience while remaining adaptable to future advances in AI technology.